

KPC Secure AI Tooling Pipeline

KPC Secure AI Tooling Pipeline — Project Documentation

Team Apex Innovators Capstone Project — Domain E: HSE, Compliance & Deployment Readiness

1. Overview

KPC’s depot, pipeline, and maintenance teams generate large volumes of operational data every day — SCADA pressure/flow readings, staff and vehicle identifiers, GPS/site coordinates — but there is no automated way to clean it, protect sensitive information within it, and deliver it safely into analytics tools. This creates compliance risk, slows down operational decisions, and costs analysts hours each week on manual data cleanup.

This project is a secure, deployable AI-tooling layer that:

1. **Ingests** simulated event-driven SCADA/depot streams.
2. **Cleans** them through automated quality gates (completeness, range, duplicates).
3. **Anonymizes** sensitive fields (staff IDs, truck plates, GPS) with field masking and enforced k-anonymity.
4. **Alerts** on both reactive threshold breaches and predictive early-warning signals (Isolation Forest anomaly detection + trend forecasting).
5. **Ships** through a gated CI/CD pipeline with real test gates, security scanning, a live canary deploy, and automatic rollback.
6. **Presents** all of the above in an executive-facing Streamlit dashboard.

2. Architecture & Modules

Module	Responsibility
<code>ingestion_quality_gates.py</code>	Simulates the raw SCADA/depot stream and runs it through three quality gates: completeness (missing transaction ID), range (pressure $\leq$ 200 PSI), and duplicate removal. Reports pass rate and a per-gate failure breakdown.
<code>anonymization_engine.py</code>	Masks direct identifiers (staff ID $\rightarrow$ SHA-256 hash, truck plate $\rightarrow$ partial redaction) and enforces k-anonymity ($k \geq 5$) on generalized GPS coordinates, suppressing any group below the threshold.
<code>predictive_model.py</code>	Trains an Isolation Forest per cycle to flag statistically anomalous pressure readings that are still inside the hard range gate (early drift signal), and fits a rolling linear trend to forecast whether pressure is on track to breach the hard gate in the next few readings.
<code>alerting_engine.py</code>	Evaluates quality/privacy metrics against operational thresholds (reactive) and the predictive model’s outputs (proactive), emitting a shared alert schema (<code>level</code> , <code>domain</code> , <code>message</code> , <code>action_required</code>) consumed by the dashboard.
<code>app.py</code>	Streamlit dashboard: Executive Summary (ROI, active alerts), Pipeline Telemetry (live charts, gate failure breakdown), Privacy & Compliance (k-anonymity distribution, anonymized payload sample).
<code>.github/workflows/deploy.yml</code>	CI/CD: pytest test gates $\rightarrow$ bandit security scan $\rightarrow$ Docker build $\rightarrow$ live canary deploy with health check $\rightarrow$ promote to <code>:stable</code> $\rightarrow$ automatic rollback on failure.

Data flow



3. CI/CD Pipeline

The pipeline is fully gated — a failure at any stage blocks the next one:

Stage	What it does
Test Gate	Runs 30 real <code>pytest</code> assertions across all four backend modules (gate logic, masking/k-anonymity correctness, alert threshold behavior, anomaly/forecast behavior), including regression checks tied directly to the proposal's own success metrics ($\geq 95\%$ pass rate, $< 10\%$ noisy alert rate, $\leq 5\%$ suppression rate).
Security Scan	<code>bandit</code> static analysis of the full codebase; fails on medium+ severity findings.
Build	Builds a Docker image (Streamlit app + all dependencies) and pushes it to GHCR, tagged with the commit SHA and <code>:canary</code> .
Canary Deploy	Runs the new image as a live container and polls Streamlit's real <code>/_stcore/health</code> endpoint plus a landing-page smoke test. This is an actual runtime check, not a placeholder.
Promote	Only reached if the canary passed. Re-tags the image <code>:stable</code> , preserving the prior stable image as <code>:previous-stable</code> .
Rollback	Fires automatically on any canary or promotion failure. Confirms <code>:stable</code> still points at the last known-good build (since traffic is never moved to a failing canary) and opens a tracking issue linking the failed run.

Full details and local run instructions are in `README.md`.

4. How This Maps to the Evaluation Criteria

Business / Social Value — Does it solve a real, existing problem?

Yes. The problem statement is a named, current operational gap at KPC: operational data (SCADA feeds, depot transaction logs) is generated continuously but has no automated path to being cleaned, protected, and safely delivered to analytics — so it either sits unused or gets processed manually.

- **Compliance risk is real and specific:** staff identifiers, vehicle plates, and site GPS coordinates are sensitive under KPC's data-handling rules; leaving them in raw analytics feeds is a live exposure, not a hypothetical one.
- **The cost is quantified, not asserted.** The dashboard's Executive Summary tab computes analyst-hours reclaimed and estimated annual cost avoidance directly from this cycle's actual fail rate — not a fixed placeholder number — so the business case updates as the pipeline's real performance changes, and is explicitly labeled as needing stakeholder-confirmed inputs before being presented as a final figure.
- **Safety angle:** the predictive layer (anomaly detection + breach forecasting) targets a genuine HSE concern — sensor drift or pressure excursions that a static threshold alone would only catch after the fact.

Technical Complexity — Did you integrate the right data sources, or build a clever ML/automation model?

- **Two-layer alerting, not one:** a reactive layer (hard thresholds on pass rate and suppression rate) and a genuinely predictive layer — an Isolation Forest trained per-cycle on pressure + local rolling-mean deviation, plus an OLS trend forecast that estimates *how many readings until a hard-gate breach*, not just whether one has already happened. The predictive layer is explicitly designed to fire *before* the reactive layer would.
- **Privacy engineering is enforced, not cosmetic:** k-anonymity ($k \geq 5$) is computed by generalizing GPS to 1-decimal resolution and structurally suppressing any group below the threshold — it's a real group-size computation with measurable suppression metrics, not a masked-column checkbox.
- **The CI/CD pipeline is itself non-trivial engineering:** 30 unit tests with deterministic, hand-built inputs (not just “does it run”); a Docker image with a real health check; a canary release that runs as an actual container the pipeline polls; and an automatic rollback path that only activates on real failure. This is closer to a production deployment pattern than a one-off demo script.

Usability / Design — Is the interface or insight easy for an operational team to use and understand immediately?

- **Three tabs, one for each audience:** Executive Summary (ROI and active alerts, for decision-makers), Pipeline Telemetry (live charts and gate failure breakdown, for data engineers), Privacy & Compliance (k-anonymity distribution and sample anonymized output, for compliance/ML reviewers).
- **Alerts are plain-language, not raw metrics.** Every alert includes a `message` and an explicit `action_required` field (“Inspect SCADA feeds for sensor degradation,” “Review geographic distribution of operations”) — an operator doesn't need to interpret a number, they're told what to do next.
- **Predictive alerts are framed in operator terms:** “pressure is climbing at X PSI per reading and will hit the limit in ~Y readings” is deliberately a plain linear-trend explanation rather than a black-box model score, because the proposal treats explainability to a non-technical stakeholder as a design requirement, not an afterthought.
- **Visual status is immediate:** color-coded scatter plot (normal / anomalous / out-of-range), a k-anonymity bar chart with the threshold line drawn on it, and a gate-failure pie chart all let someone assess pipeline health at a glance without reading logs.

Feasibility — Could this realistically move beyond a prototype into adoption, and a horizontal market strategy?

- **Built on standard, production-grade patterns already** — CI/CD gates, canary deploys with automatic rollback, audit-style anonymization metrics — rather than a one-off notebook, which is explicitly called out as the intended feasibility story.
- **Clear, honest Phase 1 → Phase 2 path:** Phase 1 uses structurally realistic synthetic data to prove the pipeline logic; Phase 2 is scoped as swapping in real or real-shaped KPC feeds once access is available, without changing the architecture.
- **The CI/CD layer generalizes beyond this dataset.** The test-gate/security-scan/canary/rollback pattern is dataset-agnostic — it would apply to any KPC operational stream (maintenance logs, fuel reconciliation, fleet telemetry), which supports a horizontal rollout across HSE/Compliance use cases rather than a single-pipeline tool.
- **Known, named gap for adoption:** the current release target is a live container the CI pipeline itself deploys and health-checks — it is not yet wired to actual KPC production infrastructure. That's the correct scope for a capstone project, and the honest next step (connecting `:stable` promotion to a real KPC deployment target) is a scoping/access question, not a redesign.

5. Success Metrics (from the proposal, now enforced as CI regression checks)

Metric	Target	Where it's enforced
Data quality pass rate	$\geq 95\%$	<code>alerting_engine.THRESHOLDS</code> ; regression-

		tested in tests/test_ingestion_quality_gates.py
Anonymization suppression rate	≤ 5%	alerting_engine.THRESHOLDS ; regression-tested in tests/test_anonymization_engine.py
Noisy (non-actionable) alert rate	< 10%	Regression-tested in tests/test_alerting_engine.py
Deployment reliability	≥ 99.5% uptime (target)	Enforced structurally via canary health-check gating before promotion, plus automatic rollback

Team Apex Innovators — Capstone Project — Domain E: HSE, Compliance & Deployment Readiness